

Modern React Frontend Stack

A learning roadmap and quick reference

Tailwind CSS

Next.js

Vite

TanStack Query

Redux Toolkit

Structured around the roadmap.sh React and Frontend roadmaps, with current details checked against each project's official docs.

Compiled September 20, 2026

1. The big picture

This guide covers five tools that show up together in most modern React projects. It follows the order of the [roadmap.sh React roadmap](#) (with the [Frontend roadmap](#) as the wider context), then adds what the official documentation says today.

How the roadmap.sh React roadmap is laid out

Roadmap section	What it covers	Where it lands in this guide
Prerequisites	The beginner JavaScript topics, before touching React	Assumed
Components and hooks	JSX, props vs state, lists and keys, refs, events, useState, useEffect, custom hooks, useReducer, useContext, useMemo, useCallback	Assumed
Build tooling	Vite	Chapter 3
Routers	React Router, TanStack Router	Next.js routing (Ch. 5)
State management	Context, Zustand, Jotai	Redux Toolkit as an established alternative (Ch. 7)
Writing CSS	CSS Modules, Tailwind CSS, Panda CSS, headless and component libraries (Radix UI, shadcn/ui, Chakra, MUI, and others)	Tailwind CSS (Ch. 2)
API calls	REST, GraphQL clients, react-query, swr, axios, rtk-query	TanStack Query and RTK Query (Ch. 6, 7)
Testing	Vitest, Jest, React Testing Library, Cypress, Playwright	Next steps (Ch. 9)
Frameworks	Next.js, Astro, React Router	Next.js (Ch. 5)
Forms, types, validation	React Hook Form, Formik, TypeScript, Zod	Next steps (Ch. 9)
Advanced	Animation, Suspense, portals, error boundaries, React Native	Next steps (Ch. 9)

Note on Redux. In the text I could extract from the roadmap.sh React roadmap, the State Management group lists Context, Zustand and Jotai, and rtk-query appears under API Calls. Redux Toolkit is still widely used, especially in existing codebases, so it is included here as an established option rather than as a roadmap.sh pick.

What each tool does

Tool	Role	Layer	Current line (Sept 2026)
Vite	Dev server and bundler for client-side apps	Tooling	Vite 8, bundled with Rolldown (released March 2026)
Tailwind CSS	Utility-first styling	UI	v4, CSS-first configuration
Next.js	Full-stack React framework (routing, server rendering, caching)	Framework	16.3.x (docs list 16.3.5)
TanStack Query	Fetching, caching and syncing server data	Server state	v5
Redux Toolkit	Predictable client-side global state, plus RTK Query	Client state	RTK 2.x (Redux 5, React-Redux 9)

Version lines come from the official docs and release posts. Patch versions move quickly, so check the docs before pinning anything. I could not confirm the newest minor release of Tailwind, TanStack Query or Redux Toolkit, so only major versions are stated for those.

2. Suggested learning order

roadmap.sh treats its order as flexible. The sequence below is my own suggestion for these five tools: start with the tools that add the fewest new ideas, and leave the framework until the concepts under it are familiar.

Step	Tool	Time	Why here
0	React fundamentals	as needed	Components, props, state, useEffect. Everything else builds on these.
1	Vite	1 to 2 days	A plain single-page app with no framework magic. Good sandbox for every later step.
2	Tailwind CSS	3 to 5 days	Style what you build in step 1. Repetition of the utility classes is the learning.
3	TanStack Query	about 1 week	Learn server state first: fetching, caching, invalidation, mutations.
4	Redux Toolkit	about 1 week	Learn client state second, once you can tell it apart from server state.
5	Next.js	2 to 3 weeks	Adds routing, server components and caching on top of everything above.
6	Combine	1 to 2 weeks	Build one project with the full stack (see Chapter 9).

A rule that saves confusion

Server state is data that lives on a server and that your UI borrows (users, products, orders). It goes stale and needs refetching. Use **TanStack Query** (or RTK Query). **Client state** is data that only exists in the browser (open modals, wizard steps, a draft, a theme). Use **useState/useReducer** first, then **Redux Toolkit** (or Zustand) when many distant components need it. Putting fetched data into Redux by hand is the most common way to end up rebuilding a worse TanStack Query.

2. Tailwind CSS

Tailwind CSS

Utility-first CSS, configured in CSS

What it is. A styling approach where you compose small single-purpose classes (flex, p-4, text-lg) directly in your markup. On the roadmap.sh React roadmap it sits under *Writing CSS*, next to CSS Modules and component libraries.

Where it stands today

- v4 replaced `tailwind.config.js` with **CSS-first configuration**: you declare design tokens as CSS variables inside an `@theme` block.
- Setup is one line, `@import "tailwindcss";`, and template files are detected automatically, so no content array.
- There is a first-party Vite plugin (`@tailwindcss/vite`) and a PostCSS plugin (`@tailwindcss/postcss`) for frameworks like Next.js.
- It relies on modern CSS (cascade layers, `@property`, `color-mix()`) and targets recent browsers: Safari 16.4+, Chrome 111+, Firefox 128+. If you must support older browsers, stay on v3.

Learn in this order

- 1 The utility model: spacing, sizing, typography, colors.
- 2 Layout: flex, grid, container queries.
- 3 Responsive design with breakpoint prefixes (md:, lg:) and state variants (hover:, focus:, dark:).
- 4 Customizing the theme with `@theme`.
- 5 Extracting repetition: React components first, `@apply` only when needed.
- 6 Pairing with a headless or copy-paste component library such as Radix UI or shadcn/ui (both listed on the roadmap).

Setup with Vite, plus a custom theme

```
# Vite project
npm install tailwindcss @tailwindcss/vite

// vite.config.ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({ plugins: [react(), tailwindcss()] })

/* src/index.css */
@import "tailwindcss";

@theme {
  --color-brand: oklch(0.6 0.2 260);
  --font-display: "Satoshi", sans-serif;
  --breakpoint-3xl: 1920px;
}
```

A component styled with utilities (tokens like text-brand come from @theme)

```
export function Card({ title, children }: { title: string; children: React.ReactNode }) {
  return (
    <div className="rounded-xl border border-gray-200 p-4 shadow-sm hover:shadow-md
      md:p-6 dark:border-gray-700 dark:bg-gray-900">
      <h2 className="text-lg font-semibold text-brand">{title}</h2>
      <div className="mt-2 text-sm text-gray-600 dark:text-gray-300">{children}</div>
    </div>
  )
}
```

Common pitfalls

- Theme namespaces matter: use --font-*, not --font-family-*, or the utility silently is not generated.
- Building class names by string concatenation ("text-" + color) defeats automatic detection. Write out complete class names.
- Upgrading from v3: run the official upgrade tool and expect changes to default border and ring styles.

Docs: <https://tailwindcss.com/docs>

3. Vite

Vite

Fast dev server and build tool for client-side apps

What it is. A development server and production bundler. It is the tool the roadmap.sh React roadmap places right after the prerequisites, and it is what you use to run a React app that is not inside a framework like Next.js.

Where it stands today

- **Vite 8** went stable on March 12, 2026. It ships **Rolldown**, a Rust-based bundler, as a single bundler, replacing the earlier esbuild (dev) plus Rollup (build) pairing. The Vite team reports builds up to 10 to 30 times faster, with plugin compatibility maintained.
- Later 8.0.x releases refer to `rolldownOptions` instead of the deprecated `rollupOptions` in messages, so expect the config name to shift.
- Create React App is no longer recommended for new projects. Vite is the usual replacement.

Learn in this order

- 1 Scaffold a project and read the generated files: `index.html`, `main.tsx`, `vite.config.ts`.
- 2 Dev server and hot module replacement (HMR): what updates without a full reload.
- 3 Environment variables: only names starting with `VITE_` are exposed to client code.
- 4 Path aliases and TypeScript config.
- 5 Plugins: the React plugin, and the Tailwind plugin from Chapter 2.
- 6 `vite build` and `vite preview`, then deploying the static output.
- 7 Testing with Vitest, which reuses your Vite config (Vitest is listed on the roadmap).

Create and run a project

```
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev          # dev server with HMR
npm run build        # production bundle in dist/
npm run preview      # serve the production build locally
```

A typical config: alias, dev proxy, env variable

```
// vite.config.ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'node:path'

export default defineConfig({
  plugins: [react()],
  resolve: { alias: { '@': path.resolve(__dirname, 'src') } },
  server: { proxy: { '/api': 'http://localhost:3000' } },
})

// use in code
const apiBase = import.meta.env.VITE_API_URL
```

Common pitfalls

- Anything prefixed `VITE_` ends up in the browser bundle. Never put secrets there.
- Vite is client-side by default. If you need server rendering, SEO-critical pages or API routes, that is the point to look at Next.js.

- When upgrading major versions, read the migration guide first, since the bundler change affects some custom build config.

Docs: <https://vite.dev/guide/>

4. TanStack Query

TanStack Query

Server state: fetching, caching, syncing

What it is. A library (formerly React Query) that manages data your app gets from a server. It handles caching, deduplicating requests, background refetching, loading and error states, and mutations. On the [roadmap.sh](#) React roadmap it appears as *react-query* under *API Calls*, alongside *swr*, *axios*, and GraphQL clients.

Where it stands today

- **v5** uses a single object argument everywhere: `useQuery({ queryKey, queryFn })`, `useMutation({ mutationFn })`. The older positional forms are gone.
- Suspense support is stable with dedicated hooks: `useSuspenseQuery`, `useSuspenseInfiniteQuery`, `useSuspenseQueries`.
- Infinite queries have a `maxPages` option to cap memory use and refetch cost.
- The `queryOptions` helper lets you define a query once and reuse it for hooks, prefetching and cache reads with full type inference.

Learn in this order

- 1 `QueryClient` and `QueryClientProvider`.
- 2 `useQuery`: query keys, status, `isPending`, `isFetching`, error.
- 3 Freshness: `staleTime` vs `gcTime` (the renamed cache time).
- 4 `useMutation` and invalidating queries afterward.
- 5 Dependent queries with `enabled`, pagination, and infinite queries.
- 6 Optimistic updates, prefetching, and Suspense hooks.
- 7 Server rendering: hydration and prefetching with Next.js.

Fetch a list and add to it

```
import { QueryClient, QueryClientProvider, useQuery, useMutation, useQueryClient }
  from '@tanstack/react-query'

const queryClient = new QueryClient()

export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Todos />
    </QueryClientProvider>
  )
}

function Todos() {
  const qc = useQueryClient()

  const { data, isPending, error } = useQuery({
    queryKey: ['todos'],
    queryFn: () => fetch('/api/todos').then(r => r.json()),
    staleTime: 30_000,
  })

  const addTodo = useMutation({
    mutationFn: (text: string) =>
      fetch('/api/todos', { method: 'POST', body: JSON.stringify({ text }) }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['todos'] }),
  })

  if (isPending) return <p>Loading...</p>
  if (error) return <p>Something went wrong</p>
  return <ul>{data.map((t: any) => <li key={t.id}>{t.text}</li>)}</ul>
}
```

Common pitfalls

- Treating `isLoading` and `isPending` as the same thing. In v5, `isPending` means no data yet; `isLoading` means pending and fetching.
- Copying query data into `useState` or `Redux`. Read it from the query instead so it stays fresh.
- Inconsistent query keys. Include every variable the query depends on, for example `['todo', id]`, so caching and invalidation work.
- Mixing v4 tutorials with v5 code. Check the version in any tutorial before copying.

Docs: <https://tanstack.com/query/latest>

5. Redux Toolkit

Redux Toolkit

Predictable global client state, plus RTK Query

What it is. The official, recommended way to write Redux. It provides `configureStore`, `createSlice`, `createAsyncThunk`, and `createEntityAdapter`, uses Immer so you can write "mutating" update code that stays immutable, and includes **RTK Query** for data fetching. [roadmap.sh](#) lists `rtk-query` under API Calls.

Where it stands today

- **RTK 2.x** ships with Redux core 5, React-Redux 9, Reselect 5 and Redux Thunk 3. You do not install those separately.
- The object syntax for `createSlice.extraReducers` and `createReducer` was removed. Use the builder callback.
- The middleware and enhancers options of `configureStore` now take callbacks.
- New in 2.0: `combineSlices` (with lazy-loaded reducers), dynamic middleware, and a callback syntax for `createSlice.reducers`.
- Many teams now reach for lighter tools (Zustand, or Context plus TanStack Query) on new small projects, while Redux Toolkit stays common in larger and existing apps.

Learn in this order

- 1 Why a store: one-way data flow, actions, reducers. Skim it; RTK hides most of the ceremony.
- 2 `configureStore` and `createSlice`.
- 3 Typed hooks and the Provider.
- 4 Selectors and keeping state normalized (`createEntityAdapter`).
- 5 `createAsyncThunk` for async logic, then RTK Query instead for most fetching.
- 6 RTK Query: `createApi`, endpoints, tags for cache invalidation, generated hooks.
- 7 Redux DevTools for time-travel debugging.

Slice, store and typed hooks

```
// store.ts
import { configureStore, createSlice, type PayloadAction } from '@reduxjs/toolkit'

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] as string[] },
  reducers: {
    added(state, action: PayloadAction<string>) { state.items.push(action.payload) },
    cleared(state) { state.items = [] },
  },
})
export const { added, cleared } = cartSlice.actions

export const store = configureStore({ reducer: { cart: cartSlice.reducer } })
export type RootState = ReturnType<typeof store.getState>
export type AppDispatch = typeof store.dispatch

// hooks.ts
import { useDispatch, useSelector } from 'react-redux'
export const useAppDispatch = useDispatch.withTypes<AppDispatch>()
export const useAppSelector = useSelector.withTypes<RootState>()
```

RTK Query endpoint definitions

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react'

export const api = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  tagTypes: ['Todo'],
  endpoints: (build) => ({
    getTodos: build.query<Todo[], void>({ query: () => 'todos', providesTags: ['Todo'] }),
    addTodo: build.mutation<Todo, string>({
      query: (text) => ({ url: 'todos', method: 'POST', body: { text } }),
      invalidatesTags: ['Todo'],
    }),
  })
})
export const { useGetTodosQuery, useAddTodoMutation } = api

// in configureStore: reducer: { [api.reducerPath]: api.reducer },
//                       middleware: (gdm) => gdm().concat(api.middleware)
```

Common pitfalls

- Storing everything in Redux. Keep local UI state in components and server data in a query cache.
- Using pre-2.0 tutorials that show `extraReducers` as an object, or the deprecated `createStore`.
- Forgetting to add the RTK Query middleware to the store, which quietly breaks caching and invalidation.
- In Next.js, creating one store at module level. On the server that shares state between users; create the store per request instead.

Docs: <https://redux-toolkit.js.org>

6. Next.js

Next.js

Full-stack React framework

What it is. A framework built on React that adds file-based routing, server rendering, data fetching on the server, API endpoints and build optimization. The roadmap.sh React roadmap lists it first under *Frameworks*, with Astro and React Router as alternatives.

Where it stands today

- The **App Router** (the `app/` directory) is the standard. It is built on React Server Components, nested layouts, streaming and Server Actions.
- **Next.js 16** introduced **Cache Components**: data is dynamic by default, and you opt in to caching with the `"use cache"` directive plus `cacheLife` and `cacheTag`. This is the reverse of the older implicit caching. Enable it with `cacheComponents: true` in `next.config.ts`.
- Turbopack is stable, and `middleware.ts` was renamed to `proxy.ts`.
- **16.3** (August 2026) adds client-side caching for the App Router and opt-in **Instant Navigations**, and lowers dev-server memory use. The docs list 16.3.5 as latest.
- Watch the version of any tutorial: older ones describe `fetch` cache options and `unstable_cache`, which the newer model replaces. The docs keep a "previous model" guide for projects not using Cache Components.

Learn in this order

- 1 Project structure: `app/`, `page.tsx`, `layout.tsx`, dynamic segments like `[id]`, `Link`.
- 2 Server Components vs Client Components: default to server, add `"use client"` only for state, effects and event handlers.
- 3 Fetching data in Server Components, plus `loading.tsx`, `error.tsx` and Suspense streaming.
- 4 Server Actions for forms and mutations.
- 5 Route handlers (`route.ts`) for API endpoints.
- 6 Caching with Cache Components: `"use cache"`, `cacheLife`, `cacheTag`, revalidation.
- 7 Metadata and SEO, images and fonts, environment variables, deployment.

Create a project and enable Cache Components

```
npx create-next-app@latest my-app # asks about TypeScript, Tailwind, App Router

// next.config.ts
import type { NextConfig } from 'next'
const nextConfig: NextConfig = { cacheComponents: true }
export default nextConfig
```

Cached server data with a small client component

```
// app/products/page.tsx (a Server Component by default)
import { cacheLife } from 'next/cache'
import { AddToCart } from './add-to-cart'

async function getProducts() {
  'use cache'
  cacheLife('hours')
  const res = await fetch('https://api.example.com/products')
  return res.json()
}

export default async function ProductsPage() {
  const products = await getProducts()
  return (
    <ul>
      {products.map((p: any) => (
        <li key={p.id}>{p.name} <AddToCart id={p.id} /></li>
      ))}
    </ul>
  )
}

// app/products/add-to-cart.tsx
'use client'
import { useState } from 'react'
export function AddToCart({ id }: { id: string }) {
  const [added, setAdded] = useState(false)
  return <button onClick={() => setAdded(true)}>{added ? 'Added' : 'Add'}</button>
}
```

Common pitfalls

- Adding "use client" to everything. It pulls the component and its imports into the client bundle.
- Using browser-only APIs (window, localStorage) in Server Components.
- Passing non-serializable values (functions, class instances) from Server to Client Components.
- Assuming a page is cached. With Cache Components, dynamic is the default and caching is explicit.

Docs: <https://nextjs.org/docs>

7. How they fit together

Two common recipes. Both use the same mental split: server state in a query cache, client state in components or a store.

	Recipe A: single-page app	Recipe B: Next.js app
Build and run	Vite 8	Next.js (Turbopack)
Routing	React Router or TanStack Router	App Router (file-based)
Styling	Tailwind CSS via @tailwindcss/vite	Tailwind CSS via @tailwindcss/postcss (create-next-app can set this up)
Server data	TanStack Query (or RTK Query)	Server Components and "use cache" for most reads; TanStack Query in Client Components for live or interactive data
Client state	useState, Context, Redux Toolkit if the app is large	Same; if using Redux, create the store per request
Good for	Dashboards, internal tools, apps behind a login	Public sites, SEO-sensitive pages, mixed static and dynamic content

TanStack Query vs RTK Query

	TanStack Query	RTK Query
Needs Redux	No	Yes, it lives in the Redux store
Best when	You want a focused data-fetching tool that works with any state library	You already use Redux Toolkit and want fetching integrated with it
Cache invalidation	By query key (invalidateQueries)	By tags (providesTags / invalidatesTags)
Advice	Pick one for server state in a given app; do not run both on the same data	

8. Practice projects

Project	Skills it exercises
Personal dashboard (Vite + Tailwind)	Tooling, responsive layout, dark mode, theme tokens with @theme
GitHub repo explorer	TanStack Query: search, pagination, caching, error and loading states
Shopping cart with checkout steps	Redux Toolkit: slices, selectors, persistence; RTK Query for the product list
Blog or docs site (Next.js)	App Router, layouts, dynamic routes, metadata, "use cache"
Full-stack capstone	Next.js + Tailwind + TanStack Query for interactive parts + Redux Toolkit only where it earns its place

roadmap.sh also keeps a project list for each roadmap: <https://roadmap.sh/react/projects>.

9. What to learn next

These come from the [roadmap.sh React roadmap](#) and are not covered in detail here.

Area	Roadmap.sh options
Testing	Vitest or Jest, React Testing Library, Cypress, Playwright
Forms	React Hook Form, Formik
Types and validation	TypeScript, Zod
Component libraries	Radix UI, shadcn/ui, React Aria, Ark UI, Chakra UI, Material UI, Panda CSS
Other state options	Zustand, Jotai, MobX, Context
Advanced React	Suspense, portals, error boundaries, animation (Framer Motion, react-spring)
Mobile	React Native

Progress checklist

☐	Scaffolded a Vite React + TypeScript project and deployed the build
☐	Styled a responsive page with Tailwind and defined a custom theme token
☐	Fetches, caches and mutated data with TanStack Query, including invalidation
☐	Built a Redux Toolkit slice with typed hooks and one RTK Query endpoint
☐	Explained Server vs Client Components without notes
☐	Used "use cache" in a Next.js page and explained what is and is not cached
☐	Built one project that combines at least three of the five tools

Sources

- [roadmap.sh React roadmap](#)
- [roadmap.sh React roadmap \(PDF version\)](#)
- [roadmap.sh Frontend roadmap](#)
- [Next.js 16 announcement](#)
- [Next.js docs: Caching with Cache Components](#)
- [Appwrite: Next.js 16.3 overview](#)
- [Vite 8 announcement](#)
- [Tailwind CSS v4.0 announcement](#)
- [Tailwind v4 migration overview \(DesignRevision\)](#)
- [TanStack Query: migrating to v5](#)
- [TanStack Query: useQuery reference](#)
- [Redux Toolkit: migrating to RTK 2.0](#)
- [Redux Toolkit: RTK Query overview](#)
- [Robin Wieruch: React libraries for 2026](#)